Lecture 05: Priority Queues — Scheduling, Ranking, and Real Applications

Topics

- Real-world problems that demand priority ordering
- The Priority Queue ADT: interface and contract
- Python's `heapq` module as a stdlib PQ
- Leaderboards, scheduling, and event-driven simulation

Goals

- Identify when a plain queue is not enough
- Use `heapq` to implement insert, pop-min, and peek correctly
- Apply PQ patterns to realistic Spotify-style examples
- Handle tie-breaking and max-heap scenarios

Roadmap

First half (≈45 min)

- Why priority queues? Real use cases first
- PQ ADT: interface and contract
- `heapq` essentials: push, pop, peek, tie-breaking
- Live demo: task scheduler
- In-class exercise 1 (commit required)

Break (3 min)

Second half (≈45 min)

- Max-heap and the negation trick
- Live demo: Spotify leaderboard with `heapq.nlargest`
- Event-driven simulation pattern
- In-class exercise 2 (commit required)
- Wrap-up and complexity checkpoints

The core problem: not everything is equal

A regular queue processes items in **arrival order** (FIFO).

But in the real world, some things are more urgent than others:

Scenario	What arrives	What we want next
Hospital ER	Patients by arrival	Most critical patient
OS scheduler	Tasks by submission	Highest-priority task
Spotify "Up Next"	Songs in queue	Song with best match score
CI/CD pipeline	Build jobs	Blocking deploys first
Game event loop	Events by time	Soonest event

A priority queue solves this. It always gives you the item with the highest (or lowest) priority, regardless of insertion order.

Real use case 1: Task scheduling

A CI/CD pipeline receives build jobs. Some block production deploys (priority 1). Others are background tasks (priority 5).

If you process by arrival (FIFO), a low-priority batch job delays a production fix.

```
Arrival order:    [report (5), security-fix (1), etl (4), incident (1)]
FIFO order:       report → security-fix → etl → incident  ← WRONG
Priority order:   security-fix → incident → etl → report  ← RIGHT
```

A priority queue processes in priority order without re-sorting every time a new job arrives.

Real use case 2: Leaderboards and ranking

Spotify wants to show the **top 10 most-played tracks** from a stream of hundreds of thousands of play events.

Naive approach: sort all plays → O(n log n) every time you want the top-10.

Smarter approach: maintain a **min-heap of size 10**.

```
As each play event arrives:
  if heap.size < 10: push it
  elif play_count > heap.peek_min(): pop the minimum, push the new entry
```

Result: always know top 10 in O(log 10) = O(1) per event.

Real use case 3: Event-driven simulation

Simulators (game engines, network models, airport queues) process **events at specific times**.

Events arrive out-of-order. You always want to process the **next soonest event**.

```
Event queue (time, action):
  (0.5,  "user_play: t003")
  (1.2,  "user_skip: t001")
  (0.8,  "ad_insert")

Processing order: t=0.5, t=0.8, t=1.2  ← always the soonest
```

A PQ makes this trivial: push events with timestamp as priority, always pop the minimum.

Real use case 4: Hospital triage

Patients arrive at an ER. Each is assigned a triage score (1 = critical, 5 = non-urgent).

- A FIFO queue would treat an ankle sprain before a heart attack if the ankle arrived first.
- A priority queue always treats the most critical patient next.

This is the original motivating problem for priority queues in computer science.

Key insight: any time you need "always give me the best/worst/urgent item," you need a PQ.

The Priority Queue ADT

Key Definitions

- **Priority Queue (PQ):** a container where each item has a **priority**, and the item with the best priority is always retrieved first
- **Min-PQ:** smallest priority value comes out first (e.g., triage level 1 = most urgent)
- **Max-PQ:** largest priority value comes out first (e.g., highest play count = top-ranked)

Core interface

Operation	Behavior
`insert(item, priority)`	Add item with given priority
`pop_min()` / `pop_max()`	Remove and return the highest-priority item
`peek()`	View the top item without removing it
`is_empty()`	True if no items remain
`size()`	Number of items in the queue

PQ: contrast with other containers

Container	Removal order	Key operation cost
Stack	Last inserted	push/pop: O(1)
Queue	First inserted (FIFO)	enqueue/dequeue: O(1)
Priority Queue	**Best priority**	push: O(log n), pop: O(log n)
Sorted list	Best priority	insert: O(n), pop: O(1)

Why not just sort?

- Sorted list insert is O(n): you re-scan on every insertion
- PQ (heap) gives O(log n) insert AND O(log n) pop
- For a dynamic stream of events, heap wins decisively

Complexity Checkpoints

Priority Queue with binary heap

- `insert` (push): **O(log n)**
- `pop_min` / `pop_max`: **O(log n)**
- `peek`: **O(1)**
- `is_empty`, `size`: **O(1)**
- Space: **O(n)**

Compare to alternatives

- Unsorted list: insert O(1), pop O(n)
- Sorted list: insert O(n), pop O(1)
- Heap: both O(log n) — best general-purpose tradeoff

Python's `heapq` module

Key facts

- `heapq` implements a **min-heap** using a regular Python list
- The smallest element is always at index 0
- You manage the list; `heapq` provides the heap operations

Core operations

```python
import heapq

pq = []                              # start with an empty list
heapq.heappush(pq, (2, "etl"))       # push (priority, item)
heapq.heappush(pq, (1, "hotfix"))
heapq.heappush(pq, (3, "report"))

top = pq[0]                          # peek: O(1), does not remove
pri, item = heapq.heappop(pq)        # pop smallest: O(log n)
```

Common Pitfall

`heapq` is a **min-heap**. To get max-heap behavior, negate your priorities.

Tie-breaking with tuples

When two items have the same priority, Python compares the next element of the tuple.

```python
import heapq
from itertools import count

seq = count()   # monotonically increasing counter
pq = []

# (priority, insertion_order, item) — insertion_order breaks ties
heapq.heappush(pq, (1, next(seq), "security-fix"))
heapq.heappush(pq, (1, next(seq), "incident"))
```

Without the counter, Python would try to compare the strings directly — which works for strings but breaks for dicts or custom objects.

Pattern: always use `(priority, counter, item)` for robustness.

Live demo: task scheduler

In [ ]:

```
import heapq
from itertools import count

# Each job is (priority, name). Lower number = higher urgency.
JOBS = [
    (3, "generate-monthly-report"),
    (1, "security-patch-deploy"),
    (4, "nightly-etl-run"),
    (1, "production-incident"),
    (2, "user-data-export"),
    (5, "log-rotation"),
]

seq = count()
pq = []
for priority, name in JOBS:
    heapq.heappush(pq, (priority, next(seq), name))

print("Processing order (highest priority first):")
while pq:
    priority, _, name = heapq.heappop(pq)
    print(f"  [P{priority}] {name}")
```

Live demo: streaming top-k

Find the top 3 most-played tracks from a stream of play records, without sorting the entire list.

In [ ]:

```python
import heapq

# Simulated play counts (track_id, total_plays)
play_stream = [
    ("t001", 12), ("t002",  4), ("t003",  6), ("t004",  6),
    ("t005",  5), ("t006",  5), ("t007", 17), ("t008",  2),
    ("t009",  1), ("t010",  2), ("t011",  5), ("t012",  1),
]

K = 3
# Keep a min-heap of size K: smallest play_count at pq[0]
# Tuple: (play_count, track_id)  — note: play_count first for heap orde
pq = []
for track_id, plays in play_stream:
    if len(pq) < K:
        heapq.heappush(pq, (plays, track_id))
    elif plays > pq[0][0]:  # beats the current minimum
        heapq.heapreplace(pq, (plays, track_id))

# The heap now holds the top-3; sort descending for display
top_k = sorted(pq, reverse=True)
print(f"Top {K} tracks by play count:")
for rank, (plays, track_id) in enumerate(top_k, start=1):
    print(f"  {rank}. {track_id}: {plays} plays")
```

`heapq` convenience functions

For one-shot top-k queries, `heapq` has built-ins:

```python
import heapq

plays = [(12, "t001"), (17, "t007"), (6, "t003"), (4, "t002")]

# Top 3 by play count (largest first)
top3 = heapq.nlargest(3, plays)

# Bottom 3 by play count
bottom3 = heapq.nsmallest(3, plays)
```

When to use which

- `nlargest` / `nsmallest`: best for **one-shot** top-k on a finished list
- `heappush` / `heappop` loop: best for **streaming** data or dynamic insertions

In [ ]:

```
import heapq

plays = [
    (12, "t001"), (4, "t002"), (6, "t003"), (6, "t004"),
    (5, "t005"), (5, "t006"), (17, "t007"), (2, "t008"),
]

print("heapq.nlargest(3):")
for p, tid in heapq.nlargest(3, plays):
    print(f"  {tid}: {p} plays")

print()
print("heapq.nsmallest(3):")
for p, tid in heapq.nsmallest(3, plays):
    print(f"  {tid}: {p} plays")
```

Common Pitfalls

1. Assuming `heapq` is a max-heap

It is a **min-heap**. `heapq.heappop(pq)` always gives the *smallest* item.

2. Missing tie-break fields

If priorities are equal and items are dicts or custom objects, comparison will fail:

```
heapq.heappush(pq, (1, {"name": "a"}))  # OK
heapq.heappush(pq, (1, {"name": "b"}))  # ERROR: can't compare dicts
```

Fix: always use `(priority, counter, item)` with a `count()` tie-breaker.

3. Mutating queued items

Changing an item's priority after it's been pushed does NOT reorder the heap. Use the lazy deletion pattern (mark stale + skip on pop) if priorities change.

In-class Exercise 1 (commit required)

Topic: Priority Queue basics — scheduling

1. **Multiple choice:** Which `heapq` operation gives you the minimum element **without removing it**?

    - A. `heapq.heappop(pq)`
    - B. `pq[0]`
    - C. `heapq.nsmallest(1, pq)[0]`
    - D. Both B and C work

2. **Short answer (2–3 sentences):** Why is a heap better than a sorted list for a CI/CD job scheduler where new jobs arrive continuously?

3. **Coding task:** Implement `hospital_triage(arrivals)` in a new cell below.

    - Input: list of `(triage_level, patient_name)` tuples (1 = most urgent)
    - Output: list of `patient_name` strings in treatment order
    - Use `heapq`; break ties by arrival order (first arrived = first treated)
    - Test with at least 5 patients including a tie

4. **Commit:**

```
git add lectures/Lecture05_Hash_Performance.ipynb
git commit -m "Lecture 05 exercise 1 work"
```

In [ ]:

```python
# Exercise 1 workspace — implement and test hospital_triage here
import heapq

def hospital_triage(arrivals):

    pass

# Test it
# Expected: Bob and Diana first (both triage 1), then Charlie, Alice, Ev
```

Break (3 minutes)

- Stand up and reset posture.
- Step away from the screen.
- Return ready for max-heaps, event simulation, and leaderboards.

Max-heap: the negation trick

`heapq` only supports min-heap. To get a max-heap, **negate your priority**:

```python
import heapq

pq = []
# Push with negated priority to simulate max-heap
heapq.heappush(pq, (-17, "t007"))  # 17 plays
heapq.heappush(pq, (-12, "t001"))  # 12 plays
heapq.heappush(pq, ( -4, "t002"))  #  4 plays

neg_plays, track_id = heapq.heappop(pq)  # pops -17 (largest play
count)
print(f"{track_id}: {-neg_plays} plays")  # t007: 17 plays
```

This is a standard Python idiom. You will see it everywhere.

In [ ]:

```
import heapq

play_counts = [
    ("t001", 12), ("t002",  4), ("t003",  6),
    ("t004",  6), ("t005",  5), ("t007", 17),
]

# Build a max-heap by negating play counts
pq = []
for track_id, plays in play_counts:
    heapq.heappush(pq, (-plays, track_id))

print("Tracks in descending play count order:")
while pq:
    neg_plays, track_id = heapq.heappop(pq)
    print(f"  {track_id}: {-neg_plays} plays")
```

Event-driven simulation

A simulation processes **events** at specific timestamps. New events can be generated while processing.

Pattern

```
1. Push initial events into a PQ with (timestamp, event_type, data)
2. While PQ is not empty:
   a. Pop the earliest event
   b. Process it (may generate new events)
   c. Push new events back into the PQ
```

The PQ guarantees you always process the **soonest event**, even if events arrive out of order.

In [ ]:

```
import heapq
from itertools import count

# Simulate a simplified Spotify event log
# Events: (timestamp, seq, event_type, data)
seq = count()
events = []

raw_events = [
    (0.5,  "play",    {"user": "u01", "track": "t003"}),
    (1.2,  "skip",    {"user": "u01", "track": "t001"}),
    (0.8,  "play",    {"user": "u02", "track": "t007"}),
    (2.0,  "playlist_end", {"user": "u01"}),
    (0.3,  "play",    {"user": "u03", "track": "t002"}),
    (1.5,  "play",    {"user": "u02", "track": "t004"}),
]

for ts, etype, data in raw_events:
    heapq.heappush(events, (ts, next(seq), etype, data))

print("Processing events in timestamp order:")
while events:
    ts, _, etype, data = heapq.heappop(events)
    print(f"  t={ts:.1f}  [{etype}]  {data}")
```

Spotify leaderboard: full pipeline

Load real play data and build a live top-5 leaderboard using `heapq`.

In [ ]:

```
import csv
import heapq
from pathlib import Path
from collections import defaultdict

DATA = Path("data")

# Load tracks
with open(DATA / "tracks.csv") as f:
    tracks = {r["track_id"]: r["track_name"] for r in csv.DictReader(f)}

# Load plays and aggregate
play_totals = defaultdict(int)
with open(DATA / "plays.csv") as f:
    for row in csv.DictReader(f):
        try:
            play_totals[row["track_id"]] += int(row["play_count"])
        except (ValueError, KeyError):
            pass

# Build top-5 leaderboard using nlargest
ranked = heapq.nlargest(5, play_totals.items(), key=lambda x: x[1])

print("🎵 Spotify Top 5 — this session")
print("-" * 40)
for rank, (tid, plays) in enumerate(ranked, start=1):
    name = tracks.get(tid, tid)
    print(f"  {rank}. {name:<20} {plays:>4} plays")
```

PQ + dictionary: a powerful combo

Many real systems pair a PQ with a dict:

- **Dict:** fast lookup of current state by key
- **PQ:** fast retrieval of the "best" item

Example: deadline-aware scheduler

```
jobs = {}  # job_id -> (priority, deadline, status)
pq   = []  # heap of (priority, seq, job_id)

# Add job
jobs[job_id] = {"priority": p, "deadline": d, "active": True}
heapq.heappush(pq, (p, next(seq), job_id))

# Cancel job (lazy deletion)
jobs[job_id]["active"] = False

# Get next active job
while pq:
    p, _, job_id = heapq.heappop(pq)
    if jobs[job_id]["active"]:
        break  # found it
```

Lazy deletion avoids expensive re-heapification when items are cancelled.

In [ ]:

```
import heapq
from itertools import count

def build_scheduler():
    jobs = {}   # job_id -> {"priority": int, "active": bool, "name": st
    pq   = []   # min-heap of (priority, seq, job_id)
    seq  = count()
    next_id = count()

    def add_job(name, priority):
        jid = next(next_id)
        jobs[jid] = {"name": name, "priority": priority, "active": True}
        heapq.heappush(pq, (priority, next(seq), jid))
        return jid

    def cancel_job(jid):
        if jid in jobs:
            jobs[jid]["active"] = False

    def run_next():
        while pq:
            _, _, jid = heapq.heappop(pq)
            if jobs[jid]["active"]:
                jobs[jid]["active"] = False
                return jobs[jid]["name"]
        return None  # queue empty

    return add_job, cancel_job, run_next

add, cancel, run = build_scheduler()
```

```
add("etl-job", 4)
add("security-fix", 1)
jid = add("report", 3)
add("incident", 1)
cancel(jid)  # cancel the report before it runs

print("Execution order:")
while (job := run()) is not None:
    print(f"  → {job}")
```

In-class Exercise 2 (commit required)

Topic: Max-heap, leaderboard, and event simulation

1. **Multiple choice:** You want to find the 5 most-played tracks from a list of 10,000. Which is most efficient?

    - A. Sort all 10,000, take the last 5
    - B. `heapq.nlargest(5, plays)`
    - C. Build a max-heap and pop 5 times
    - D. B and C have equivalent complexity; B has simpler code

2. **Short answer (2–3 sentences):** Explain the lazy deletion pattern. When would you use it instead of removing an item from the middle of the heap?

3. **Coding task:** Implement `genre_leaderboard(plays, tracks, genre, top_n)` in a new cell below.

    - `plays` : list of `(track_id, play_count)` tuples
    - `tracks` : dict mapping `track_id -> {"title": str, "genre": str}`
    - Returns the top `top_n` `(track_id, play_count)` tuples for that genre, highest plays first
    - Use `heapq.nlargest` or a min-heap of size `top_n`
    - Test with at least two genres

4. **Commit:**

```
git add lectures/Lecture05_Hash_Performance.ipynb
```

In [ ]:

```python
# Exercise 2 workspace
import heapq

def genre_leaderboard(plays, tracks, genre, top_n):

    pass

# Sample data for testing
```

Wrap-up

Key Definitions revisited

- **Priority Queue ADT:** insert + pop-best + peek
- **Min-heap:** smallest priority out first; `heapq` implements this natively
- **Max-heap:** negate priorities to simulate with `heapq`
- **Lazy deletion:** mark items stale in a dict; skip them on pop

Complexity Checkpoints

- `heappush`: **O(log n)**
- `heappop`: **O(log n)**
- peek (`pq[0]`): **O(1)**
- `nlargest(k, ...)`: **O(n log k)** — faster than full sort when k << n

Common Pitfalls to remember

- `heapq` is min-heap only — negate for max
- Always use `(priority, counter, item)` to avoid comparison errors
- Mutating queued items does NOT reorder the heap

Next

- **Lecture 06:** Complexity Analysis — formally analyzing pipelines, nested loops, and amortized costs
- **Lab 05:** Hash Performance — hands-on collision simulation (see `labs/lab05/`)